Kitchener Watermains + Water Main Breaks + Soil data

Initial Pre-processing (Anh)

Purpose

In [2]:
```
#Import required packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [3]:
```
#Import Kitchener dataset
breaks = pd.read_csv("../data/Kitchener_watermainbreaks.csv")
mains = pd.read_csv("../data/Kitchener_watermains.csv")

#Quick check on features
print("\nBreaks info:")
print(breaks.info())

print("\nMains info:")
print(mains.info())
```

```
Breaks info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2994 entries, 0 to 2993
Data columns (total 37 columns):
 #   Column                                          Non-Null Count  Dtype
---  ------                                          --------------  -----
 0   OBJECTID                                        2994 non-null   int64
 1   Wat Break Incident ID                           2994 non-null   int64
 2   Incident date                                   2994 non-null   object
 3   Type of Asset Broken                            2994 non-null   object
 4   Does the road need to be closed?                540 non-null    object
 5   Does the sidewalk need to be closed?            537 non-null    object
 6   Estimated Hours for Repair                      538 non-null    object
 7   Estimated Number of Units Impacted              525 non-null    object
 8   CW Service Request Number                       25 non-null     float64
 9   Current status of the break                     2994 non-null   object
 10  Status last updated date                        2843 non-null   object
 11  CW Workorder #                                  1826 non-null   float64
 12  Date operations was returned to normal service  106 non-null    object
 13  Nature of Break                                 2682 non-null   object
 14  Apparent cause of break                         2558 non-null   object
 15  Repair Type                                     246 non-null    object
 16  Type of Planned Maintenance                     0 non-null      float64
 17  List Valves Closed                              3 non-null      object
 18  List Valves Opened                              2 non-null      float64
 19  List Hydrants Called Out                        48 non-null     object
 20  List Hydrants Called Back In                    36 non-null     object
 21  Categorization of the Break                     2693 non-null   object
 22  Road Segment ID                                 2994 non-null   int64
 23  Closest Civic Number                            2897 non-null   object
 24  Street                                          2979 non-null   object
 25  Related Asset ID                                2994 non-null   int64
 26  Related Asset Depth (m)                         15 non-null     float64
 27  Depth of Frost (m)                              82 non-null     float64
 28  Asset Size (cm)                                 2695 non-null   float64
 29  Year Asset Installed                            2691 non-null   float64
 30  Asset Material                                  2695 non-null   object
 31  Asset Exists                                    2994 non-null   object
 32  GLOBALID                                        2994 non-null   object
 33  UPDATE_BY                                       2447 non-null   object
 34  UPDATE_DATE                                     2447 non-null   object
 35  x                                               2994 non-null   float64
 36  y                                               2994 non-null   float64
dtypes: float64(10), int64(4), object(23)
memory usage: 865.6+ KB
None

Mains info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16163 entries, 0 to 16162
Data columns (total 27 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   OBJECTID              16163 non-null  int64
 1   WATMAINID             16163 non-null  int64
 2   STATUS                16163 non-null  object
 3   PRESSURE_ZONE         16163 non-null  object
 4   ROADSEGMENTID         16163 non-null  int64
 5   MAP_LABEL             16163 non-null  object
 6   CATEGORY              16163 non-null  object
 7   PIPE_SIZE             16163 non-null  int64
 8   MATERIAL              16163 non-null  object
 9   LINED                 16163 non-null  object
 10  LINED_DATE            54 non-null     object
 11  LINED_MATERIAL        16163 non-null  object
 12  INSTALLATION_DATE     16163 non-null  object
```

```
 13  ACQUISITION           16163 non-null  object
 14  CONSULTANT            4566 non-null   object
 15  OWNERSHIP             16163 non-null  object
 16  BRIDGE_MAIN           16163 non-null  object
 17  BRIDGE_DETAILS        8 non-null      object
 18  CRITICALITY           16163 non-null  int64
 19  REL_CLEANING_AREA     16163 non-null  int64
 20  REL_CLEANING_SUBAREA  16163 non-null  object
 21  Undersized            16163 non-null  object
 22  Shallow Main          16163 non-null  object
 23  Condition Score       16163 non-null  float64
 24  OVERSIZED             16163 non-null  object
 25  CLEANED               16163 non-null  object
 26  Shape__Length         16163 non-null  float64
dtypes: float64(2), int64(6), object(19)
memory usage: 3.3+ MB
None
```

In [4]:

```python
#Next, we check for ratio of missing values in each columns

#Missing ratio for each column in breaks
breaks_missing_ratio = (breaks.isnull().mean() * 100).sort_values(ascending=False)

print("Missing ratio (%) in breaks:")
print(breaks_missing_ratio)

#Missing ratio for each column in mains
mains_missing_ratio = (mains.isnull().mean() * 100).sort_values(ascending=False)

print("-----------------")
print("Missing ratio (%) in mains:")
print(mains_missing_ratio)
```

```
Missing ratio (%) in breaks:
Type of Planned Maintenance                          100.000000
List Valves Opened                                    99.933200
List Valves Closed                                    99.899800
Related Asset Depth (m)                               99.498998
CW Service Request Number                             99.164997
List Hydrants Called Back In                          98.797595
List Hydrants Called Out                              98.396794
Depth of Frost (m)                                    97.261189
Date operations was returned to normal service        96.459586
Repair Type                                           91.783567
Estimated Number of Units Impacted                    82.464930
Does the sidewalk need to be closed?                  82.064128
Estimated Hours for Repair                            82.030728
Does the road need to be closed?                      81.963928
CW Workorder #                                        39.011356
UPDATE_DATE                                           18.269873
UPDATE_BY                                             18.269873
Apparent cause of break                               14.562458
Nature of Break                                       10.420842
Year Asset Installed                                  10.120240
Categorization of the Break                           10.053440
Asset Material                                         9.986640
Asset Size (cm)                                        9.986640
Status last updated date                               5.043420
Closest Civic Number                                   3.239813
Street                                                 0.501002
GLOBALID                                               0.000000
Asset Exists                                           0.000000
x                                                      0.000000
OBJECTID                                               0.000000
Related Asset ID                                       0.000000
Road Segment ID                                        0.000000
Wat Break Incident ID                                  0.000000
Current status of the break                            0.000000
Type of Asset Broken                                   0.000000
Incident date                                          0.000000
y                                                      0.000000
dtype: float64
-----------------
Missing ratio (%) in mains:
BRIDGE_DETAILS          99.950504
LINED_DATE              99.665904
CONSULTANT              71.750294
OBJECTID                 0.000000
CLEANED                  0.000000
OVERSIZED                0.000000
Condition Score          0.000000
Shallow Main             0.000000
Undersized               0.000000
REL_CLEANING_SUBAREA     0.000000
REL_CLEANING_AREA        0.000000
CRITICALITY              0.000000
BRIDGE_MAIN              0.000000
OWNERSHIP                0.000000
ACQUISITION              0.000000
WATMAINID                0.000000
INSTALLATION_DATE        0.000000
LINED_MATERIAL           0.000000
LINED                    0.000000
MATERIAL                 0.000000
PIPE_SIZE                0.000000
CATEGORY                 0.000000
MAP_LABEL                0.000000
ROADSEGMENTID            0.000000
PRESSURE_ZONE            0.000000
```

```
STATUS                    0.000000
Shape__Length             0.000000
dtype: float64
```

In [5]:
```
#Drop columns with over 80% missing values
#plus some other columns that are not needed for analysis (like "CW Workorder #", "CO

#Define columns to be dropped
#Breaks
breaks_drop_cols = [
    "Type of Planned Maintenance",
    "List Valves Opened",
    "List Valves Closed",
    "Related Asset Depth (m)",
    "CW Service Request Number",
    "List Hydrants Called Back In",
    "List Hydrants Called Out",
    "Depth of Frost (m)",
    "Date operations was returned to normal service",
    "Repair Type",
    "Estimated Number of Units Impacted",
    "Does the sidewalk need to be closed?",
    "Estimated Hours for Repair",
    "Does the road need to be closed?",
    "CW Workorder #",
    "OBJECTID",
    "GLOBALID",
    "UPDATE_BY",
    "UPDATE_DATE"
]

#Mains
mains_drop_cols = [
    "BRIDGE_DETAILS",
    "LINED_DATE",
    "CONSULTANT",
    "OBJECTID",
    "MAP_LABEL"
]

#Drop columns in the list
breaks_clean = breaks.drop(columns=[col for col in breaks_drop_cols if col in breaks.
mains_clean = mains.drop(columns=[col for col in mains_drop_cols if col in mains.colu

#Quick check
print("Breaks data after dropping columns:", breaks_clean.shape)
print("Mains data after dropping columns:", mains_clean.shape)
```

```
Breaks data after dropping columns: (2994, 18)
Mains data after dropping columns: (16163, 22)
```

In [6]:
```
#Now we convert the date/time to correct format

#Breaks date columns
breaks_date_cols = [
    "Incident date",
    "Status last updated date"
]

for col in breaks_date_cols:
    if col in breaks_clean.columns:
        breaks_clean[col] = pd.to_datetime(breaks_clean[col], errors="coerce")

#Mains date columns
mains_date_cols = [
    "INSTALLATION_DATE"
```

```
]

for col in mains_date_cols:
    if col in mains_clean.columns:
        mains_clean[col] = pd.to_datetime(mains_clean[col], errors="coerce")

#Quick check on cleaned data
print("\nBreaks head:")
print(breaks.head())
print("-----------------")
print("\nMains head:")
print(mains.head())
```

```
Breaks head:
   OBJECTID  Wat Break Incident ID          Incident date  \
0         1                   2252   12/1/2017 3:15:00 PM
1      7874                   1311  3/26/2001 12:00:00 AM
2      7875                   1325   9/6/2006 12:00:00 AM
3      7876                   1328  9/11/2006 12:00:00 AM
4      7877                   1308  1/27/2000 12:00:00 AM

  Type of Asset Broken Does the road need to be closed?  \
0                 MAIN                 Partially Closed
1              SERVICE                              NaN
2              SERVICE                              NaN
3              SERVICE                              NaN
4              SERVICE                              NaN

  Does the sidewalk need to be closed? Estimated Hours for Repair  \
0                                 Open                12-16 hours
1                                  NaN                        NaN
2                                  NaN                        NaN
3                                  NaN                        NaN
4                                  NaN                        NaN

  Estimated Number of Units Impacted  CW Service Request Number  \
0                                 47                   123456.0
1                                NaN                        NaN
2                                NaN                        NaN
3                                NaN                        NaN
4                                NaN                        NaN

  Current status of the break  ... Depth of Frost (m)  Asset Size (cm)  \
0            REPAIR COMPLETED  ...                0.3            450.0
1            REPAIR COMPLETED  ...                NaN             13.0
2            REPAIR COMPLETED  ...                NaN             25.0
3            REPAIR COMPLETED  ...                NaN             25.0
4            REPAIR COMPLETED  ...                NaN             25.0

  Year Asset Installed Asset Material Asset Exists  \
0               1937.0             CI            Y
1               1965.0            XXX            Y
2               1967.0            XXX            Y
3               1964.0            PVC            Y
4               1967.0            XXX            Y

                              GLOBALID  UPDATE_BY            UPDATE_DATE  \
0  3521d297-1a2e-4e7b-a071-fc53ed87e965   GIS_DATA   10/3/2018 2:42:09 PM
1  72445d62-16a8-43c1-9733-56b06015b077     DAVIDV  12/15/2011 3:38:28 PM
2  3bdc8931-31c0-4090-a07a-a6847781dd97     DAVIDV  12/15/2011 3:38:28 PM
3  f75ad0b1-5b2a-4125-8ad5-2b9a037debd7     DAVIDV  12/15/2011 3:38:28 PM
4  5a3c5d03-0899-4899-95e7-278bc5cbb682     DAVIDV  12/15/2011 3:38:28 PM

             x             y
0  541741.1448  4.812354e+06
1  539253.7576  4.807875e+06
2  545329.4589  4.810392e+06
3  539592.5208  4.808291e+06
4  543897.8495  4.810175e+06

[5 rows x 37 columns]
-----------------

Mains head:
   OBJECTID  WATMAINID  STATUS PRESSURE_ZONE  ROADSEGMENTID        MAP_LABEL  \
0     77753      10080  ACTIVE         KIT 6           5457  34.2m 450mm  DI
1     77754      76299  ACTIVE         KIT 4           6390    .4m 300mm  DI
2     77755      10110  ACTIVE         KIT 6           5393  67.9m 450mm  DI
3     77756      82566  ACTIVE         KIT 6           5393     7m 450mm  DI
```

```
4     77757      82568  ACTIVE         KIT 6           5394    15m 450mm  DI

  CATEGORY  PIPE_SIZE MATERIAL LINED  ... BRIDGE_DETAILS CRITICALITY  \
0  TREATED        450       DI    NO  ...            NaN           6
1  TREATED        300       DI    NO  ...            NaN           7
2  TREATED        450       DI    NO  ...            NaN           7
3  TREATED        450       DI    NO  ...            NaN           6
4  TREATED        450       DI    NO  ...            NaN           6

  REL_CLEANING_AREA REL_CLEANING_SUBAREA Undersized Shallow Main  \
0                 6                    8          N            N
1                 1                    4          N            N
2                 6                    8          N            N
3                 6                    8          N            N
4                 6                    8          N            N

  Condition Score OVERSIZED  CLEANED  Shape__Length
0            8.50         N        N      34.210563
1            8.50         N        N       0.355118
2            6.10         N        Y      67.852910
3            4.58         N        Y       7.039328
4            8.50         N        Y      14.956131

[5 rows x 27 columns]
```

```
/var/folders/bk/sln2b49n47g24cwmlq18xqhc0000gn/T/ipykernel_7879/3242174556.py:11: User
Warning: Could not infer format, so each element will be parsed individually, falling
back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a
format.
  breaks_clean[col] = pd.to_datetime(breaks_clean[col], errors="coerce")
/var/folders/bk/sln2b49n47g24cwmlq18xqhc0000gn/T/ipykernel_7879/3242174556.py:11: User
Warning: Could not infer format, so each element will be parsed individually, falling
back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a
format.
  breaks_clean[col] = pd.to_datetime(breaks_clean[col], errors="coerce")
/var/folders/bk/sln2b49n47g24cwmlq18xqhc0000gn/T/ipykernel_7879/3242174556.py:20: User
Warning: Could not infer format, so each element will be parsed individually, falling
back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a
format.
  mains_clean[col] = pd.to_datetime(mains_clean[col], errors="coerce")
```

In [7]:

```
#Check for duplicates
print("Duplicate break IDs:", breaks_clean["Wat Break Incident ID"].duplicated().sum(
print("Duplicate main IDs:", mains_clean["WATMAINID"].duplicated().sum())
print("Duplicate road segment IDs in mains:", mains_clean["ROADSEGMENTID"].duplicated
print("Duplicate related asset IDs in breaks:", breaks_clean["Related Asset ID"].dupl
```

```
Duplicate break IDs: 0
Duplicate main IDs: 0
Duplicate road segment IDs in mains: 11155
Duplicate related asset IDs in breaks: 1481
```

So break incident ID and water main id are unique in both datasets. We will use them as uniques identifier when merging.

In [9]:

```
#Check match rate before merging
asset_match_rate = breaks_clean["Related Asset ID"].isin(mains_clean["WATMAINID"]).me
print(f"Asset ID match rate: {asset_match_rate:.2f}%")

matched_breaks = breaks_clean["Related Asset ID"].isin(mains_clean["WATMAINID"]).sum(
unmatched_breaks = (~breaks_clean["Related Asset ID"].isin(mains_clean["WATMAINID"]))

print("Matched break rows:", matched_breaks)
print("Unmatched break rows:", unmatched_breaks)
```

```
Asset ID match rate: 81.90%
Matched break rows: 2452
Unmatched break rows: 542
```

In [10]:
```python
#Filter to main breaks
breaks_main = breaks_clean[breaks_clean["Type of Asset Broken"] == "MAIN"].copy()

#Keep only breaks that match a main asset
breaks_main_matched = breaks_main[
    breaks_main["Related Asset ID"].isin(mains_clean["WATMAINID"])
].copy()

print("MAIN breaks:", breaks_main.shape)
print("Matched MAIN breaks:", breaks_main_matched.shape)
```

```
MAIN breaks: (2922, 18)
Matched MAIN breaks: (2451, 18)
```

In [11]:
```python
#Aggregate break record at pipe level
break_summary = (
    breaks_main_matched
    .groupby("Related Asset ID")
    .agg(
        break_count=("Wat Break Incident ID", "count"),
        first_break_date=("Incident date", "min"),
        last_break_date=("Incident date", "max")
    )
    .reset_index()
    .rename(columns={"Related Asset ID": "WATMAINID"})
)

break_summary["has_break"] = 1

#Quick check
display(break_summary.head())
```

	WATMAINID	break_count	first_break_date	last_break_date	has_break
0	270	2	1998-11-20	2007-01-21 00:00:00	1
1	290	4	1987-01-01	2013-02-20 00:00:00	1
2	590	1	2012-11-03	2012-11-03 00:00:00	1
3	660	2	2003-02-21	2003-02-22 00:00:00	1
4	670	3	2007-11-29	2020-12-24 23:02:50	1

In [12]:
```python
#Check the number of pipe with and withdown breaks
pipe_df = mains_clean.merge(
    break_summary,
    on="WATMAINID",
    how="left"
)

pipe_df["break_count"] = pipe_df["break_count"].fillna(0).astype(int)
pipe_df["has_break"] = pipe_df["has_break"].fillna(0).astype(int)

print("Pipe-level dataset shape:", pipe_df.shape)
print(pipe_df["has_break"].value_counts())
display(pipe_df.head())

#Quick check
print("Unique pipes with observed break:", break_summary["WATMAINID"].nunique())
print("Total pipes in mains:", mains_clean["WATMAINID"].nunique())
```

```
Pipe-level dataset shape: (16163, 26)
has_break
0    14929
1     1234
Name: count, dtype: int64
```

	WATMAINID	STATUS	PRESSURE_ZONE	ROADSEGMENTID	CATEGORY	PIPE_SIZE	MATERIAL
0	10080	ACTIVE	KIT 6	5457	TREATED	450	DI
1	76299	ACTIVE	KIT 4	6390	TREATED	300	DI
2	10110	ACTIVE	KIT 6	5393	TREATED	450	DI
3	82566	ACTIVE	KIT 6	5393	TREATED	450	DI
4	82568	ACTIVE	KIT 6	5394	TREATED	450	DI

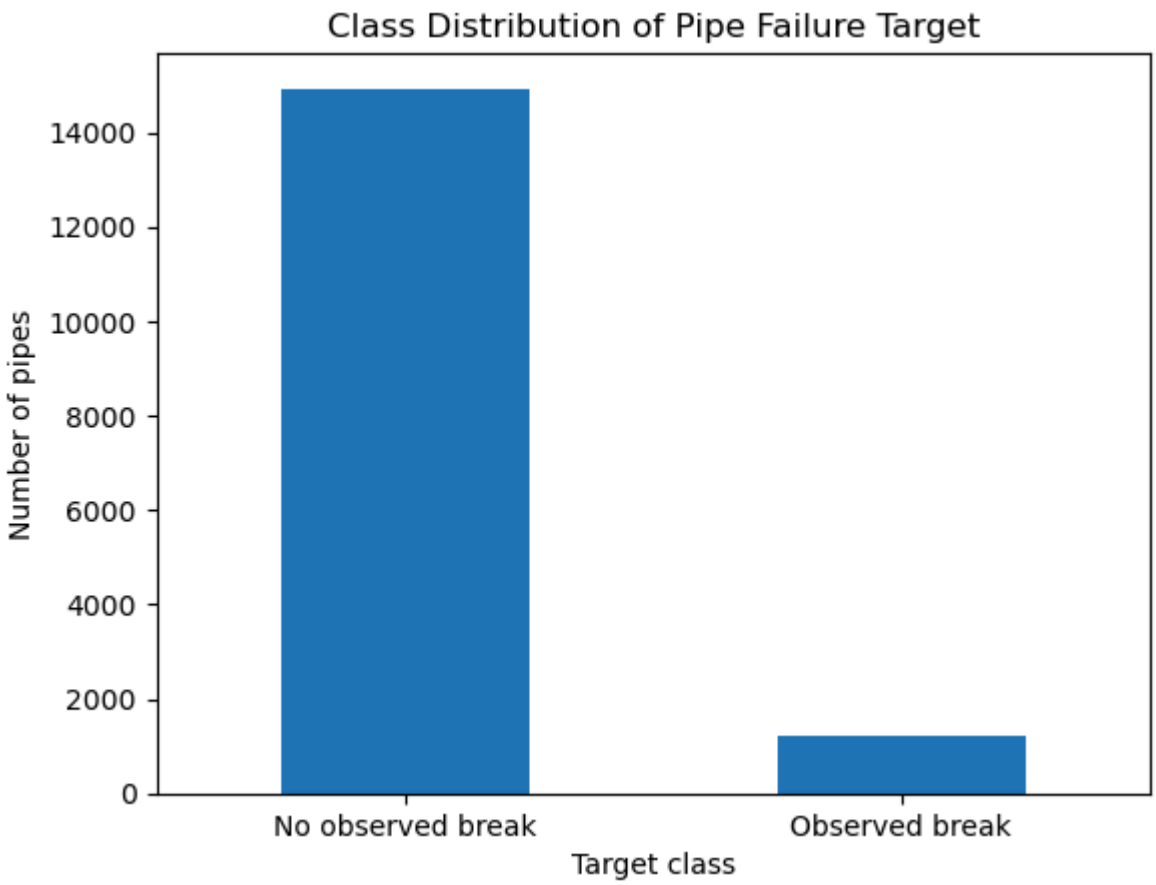
5 rows × 26 columns

```
Unique pipes with observed break: 1234
Total pipes in mains: 16163
```

From this ratio, we see that data is imbalanced.

In [14]:
```
#Explore class distribution
class_counts = pipe_df["has_break"].value_counts().sort_index()
class_pct = (pipe_df["has_break"].value_counts(normalize=True).sort_index() * 100).ro

#Visualise class imbalance
plt.figure()
class_counts.plot(kind="bar")
plt.xticks([0, 1], ["No observed break", "Observed break"], rotation=0)
plt.ylabel("Number of pipes")
plt.xlabel("Target class")
plt.title("Class Distribution of Pipe Failure Target")
plt.show()
```



In [15]:
```
import sys
!{sys.executable} -m pip install pyproj
```

```
Requirement already satisfied: pyproj in /opt/anaconda3/lib/python3.12/site-packages
(3.7.2)
Requirement already satisfied: certifi in /opt/anaconda3/lib/python3.12/site-packages
(from pyproj) (2025.1.31)
```

In [16]:
```
import pandas as pd
from pyproj import Transformer

#Copy dataset
breaks_test = breaks_clean.copy()

#Keep only valid coordinates
breaks_test = breaks_test.dropna(subset=["x", "y"]).copy()

#Create transformer: EPSG:26917 -> EPSG:4326
transformer = Transformer.from_crs("EPSG:26917", "EPSG:4326", always_xy=True)

#Convert x, y to lon, lat
breaks_test["longitude"], breaks_test["latitude"] = transformer.transform(
    breaks_test["x"].values,
    breaks_test["y"].values
)

#Show sample
display(
    breaks_test[[
        "Wat Break Incident ID",
        "Street",
        "x", "y",
        "longitude", "latitude"
    ]].head(10)
)
```

	Wat Break Incident ID	Street	x	y	longitude	latitude
0	2252	LANCASTER ST W	541741.1448	4.812354e+06	-80.484001	43.462931
1	1311	CLOVERDALE CRES	539253.7576	4.807875e+06	-80.515071	43.422734
2	1325	WREN CRES	545329.4589	4.810392e+06	-80.439808	43.445058
3	1328	GREENBROOK DR	539592.5208	4.808291e+06	-80.510856	43.426469
4	1308	MONTGOMERY RD	543897.8495	4.810175e+06	-80.457516	43.443192
5	1312	WESTHEIGHTS DR	538063.0983	4.808117e+06	-80.529762	43.424980
6	1321	RIVER RD E	547018.7513	4.808296e+06	-80.419113	43.426079
7	1326	BLEAMS RD	538558.0000	4.805205e+06	-80.523854	43.398733
8	2047	MILL ST	541546.9539	4.809431e+06	-80.486624	43.436626
9	1324	KINGSWAY DR	545344.7332	4.808249e+06	-80.439797	43.425757

In [17]:
```
breaks_test.to_csv("../data/kitchener_breaks_latlon.csv", index=False)
```

In [18]:
```
import sys
!{sys.executable} -m pip install geopandas shapely pyproj fiona
```

```
Requirement already satisfied: geopandas in /opt/anaconda3/lib/python3.12/site-package
s (1.1.3)
Requirement already satisfied: shapely in /opt/anaconda3/lib/python3.12/site-packages
(2.1.2)
Requirement already satisfied: pyproj in /opt/anaconda3/lib/python3.12/site-packages
(3.7.2)
Requirement already satisfied: fiona in /opt/anaconda3/lib/python3.12/site-packages
(1.10.1)
Requirement already satisfied: numpy>=1.24 in /opt/anaconda3/lib/python3.12/site-packa
ges (from geopandas) (1.26.4)
Requirement already satisfied: pyogrio>=0.7.2 in /opt/anaconda3/lib/python3.12/site-pa
ckages (from geopandas) (0.12.1)
Requirement already satisfied: packaging in /opt/anaconda3/lib/python3.12/site-package
s (from geopandas) (24.1)
Requirement already satisfied: pandas>=2.0.0 in /opt/anaconda3/lib/python3.12/site-pac
kages (from geopandas) (2.2.2)
Requirement already satisfied: certifi in /opt/anaconda3/lib/python3.12/site-packages
(from pyproj) (2025.1.31)
Requirement already satisfied: attrs>=19.2.0 in /opt/anaconda3/lib/python3.12/site-pac
kages (from fiona) (23.1.0)
Requirement already satisfied: click~=8.0 in /opt/anaconda3/lib/python3.12/site-packag
es (from fiona) (8.1.7)
Requirement already satisfied: click-plugins>=1.0 in /opt/anaconda3/lib/python3.12/sit
e-packages (from fiona) (1.1.1.2)
Requirement already satisfied: cligj>=0.5 in /opt/anaconda3/lib/python3.12/site-packag
es (from fiona) (0.7.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/anaconda3/lib/python3.1
2/site-packages (from pandas>=2.0.0->geopandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.12/site-pack
ages (from pandas>=2.0.0->geopandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.12/site-pa
ckages (from pandas>=2.0.0->geopandas) (2023.3)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.12/site-packages
(from python-dateutil>=2.8.2->pandas>=2.0.0->geopandas) (1.16.0)
```

In [19]:

```python
import geopandas as gpd

soil_gdf = gpd.read_file("Soil_Survey_Complex.geojson")

print(soil_gdf.shape)
print(soil_gdf.columns.tolist())
print(soil_gdf.crs)
print(soil_gdf.head())
```

```
(144947, 77)
['OGF_ID', 'MAPUNIT', 'SOIL_CMPLX', 'PERCENT1', 'SOILTYPE1', 'SOILCODE1', 'SOIL_NAME
1', 'SYMBOL1', 'PARNT_MAT1', 'LANDSCAPE1', 'SLOPE1', 'CLASS1', 'RANGE1', 'STONINESS1',
'CLI1', 'CLI1_1', 'CLI1_2', 'SURVEY1', 'DRAINAGE1', 'DR_DESIGN1', 'HYDRO1', 'ATEXTURE
1', 'MODIFIER1', 'K_FACTOR1', 'PERCENT2', 'SOILTYPE2', 'SOILCODE2', 'SOIL_NAME2', 'SYM
BOL2', 'PARNT_MAT2', 'LANDSCAPE2', 'SLOPE2', 'CLASS2', 'RANGE2', 'STONINESS2', 'CLI2',
'CLI2_1', 'CLI2_2', 'SURVEY2', 'DRAINAGE2', 'DR_DESIGN2', 'HYDRO2', 'ATEXTURE2', 'MODI
FIER2', 'K_FACTOR2', 'PERCENT3', 'SOILTYPE3', 'SOILCODE3', 'SOIL_NAME3', 'SYMBOL3', 'P
ARNT_MAT3', 'LANDSCAPE3', 'SLOPE3', 'CLASS3', 'RANGE3', 'STONINESS3', 'CLI3', 'CLI3_
1', 'CLI3_2', 'SURVEY3', 'DRAINAGE3', 'DR_DESIGN3', 'HYDRO3', 'ATEXTURE3', 'MODIFIER
3', 'K_FACTOR3', 'CLI_SYS', 'ACRES', 'HECTARES', 'LOCATION_ACCURACY', 'GEOMETRY_UPDATE
_DATETIME', 'EFFECTIVE_DATETIME', 'SYSTEM_DATETIME', 'OBJECTID', 'SHAPEAREA', 'SHAPELE
N', 'geometry']
EPSG:4326
      OGF_ID            MAPUNIT  SOIL_CMPLX  PERCENT1    SOILTYPE1 SOILCODE1  \
0  126293316     ONOND355Ccl/B0           1       100  ONCGG~~~~~A       CGG
1  126293317    ONOND355Shcl/A0           1       100  ONSHN~~~~~N       SHN
2  126293318   ONOND355Bsl=F/A0           2        50  ONBIZ~~~~~N       BIZ
3  126293319       ONOND355U/A0           1       100  ONUPK~~~~~N       UPK
4  126293310  ONOND355F=Shcl/A0           2        50  ONFEK~~~~~N       FEK

  SOIL_NAME1 SYMBOL1 PARNT_MAT1 LANDSCAPE1  ...       ACRES    HECTARES  \
0      CLEGG     Ccl       None       None  ...   278.05756   112.52590
1   SHETLAND    Shcl       None       None  ...   337.22133   136.46863
2        BIZ     Bsl       None       None  ...   586.00446   237.14759
3   UNO PARK       U       None       None  ...   154.29736    62.44192
4      FLECK       F       None       None  ...  2640.81319  1068.69918

  LOCATION_ACCURACY  GEOMETRY_UPDATE_DATETIME        EFFECTIVE_DATETIME  \
0              None 2019-09-12 09:19:46+00:00 2019-09-12 07:30:24+00:00
1              None 2019-09-12 09:25:04+00:00 2019-09-12 07:30:24+00:00
2              None 2019-09-12 09:26:32+00:00 2019-09-12 07:30:25+00:00
3              None 2019-09-12 09:31:23+00:00 2019-09-12 07:30:25+00:00
4              None 2019-09-12 09:29:44+00:00 2019-09-12 07:30:23+00:00

            SYSTEM_DATETIME OBJECTID SHAPEAREA SHAPELEN  \
0 2019-09-12 07:30:24+00:00  4309636         0        0
1 2019-09-12 07:30:24+00:00  4309637         0        0
2 2019-09-12 07:30:25+00:00  4309638         0        0
3 2019-09-12 07:30:25+00:00  4309639         0        0
4 2019-09-12 07:30:23+00:00  4309640         0        0

                                            geometry
0  POLYGON ((-80.33268 48.43092, -80.33297 48.431...
1  POLYGON ((-80.26207 48.43094, -80.26222 48.431...
2  POLYGON ((-80.02183 48.42826, -80.02342 48.428...
3  POLYGON ((-80.40647 48.42367, -80.40658 48.424...
4  POLYGON ((-80.40051 48.43119, -80.40087 48.430...

[5 rows x 77 columns]
```

In [20]:

```
soil_keep_cols = [
    "OGF_ID",
    "MAPUNIT",
    "SOIL_CMPLX",
    "PERCENT1",
    "SOIL_NAME1",
    "PARNT_MAT1",
    "SLOPE1",
    "DRAINAGE1",
    "DR_DESIGN1",
    "HYDRO1",
    "ATEXTURE1",
    "K_FACTOR1",
    "geometry"
]
```

```
soil_small = soil_gdf[soil_keep_cols].copy()
print(soil_small.shape)
soil_small.head()
```

```
(144947, 13)
```

Out[20]:

	OGF_ID	MAPUNIT	SOIL_CMPLX	PERCENT1	SOIL_NAME1	PARNT_MAT1	SLOPE
0	126293316	ONOND355Ccl/B0	1	100	CLEGG	None	3
1	126293317	ONOND355Shcl/A0	1	100	SHETLAND	None	1
2	126293318	ONOND355Bsl=F/A0	2	50	BIZ	None	1
3	126293319	ONOND355U/A0	1	100	UNO PARK	None	1
4	126293310	ONOND355F=Shcl/A0	2	50	FLECK	None	1

In [21]:

```
breaks_points = gpd.GeoDataFrame(
    breaks_test.copy(),
    geometry=gpd.points_from_xy(breaks_test["longitude"], breaks_test["latitude"]),
    crs="EPSG:4326"
)

print(breaks_points.shape)
breaks_points.head()
```

```
(2994, 21)
```

Out[21]:

	Wat Break Incident ID	Incident date	Type of Asset Broken	Current status of the break	Status last updated date	Nature of Break	Apparent cause of break	Categorization of the Break
0	2252	2017-12-01 15:15:00	MAIN	REPAIR COMPLETED	2017-12-02 00:00:00	CORROSION AND FITTING/JOINT	AGE	CATEGORY 2
1	1311	2001-03-26 00:00:00	SERVICE	REPAIR COMPLETED	2011-12-15 15:38:28	UNKNOWN	OTHER	CATEGORY 1
2	1325	2006-09-06 00:00:00	SERVICE	REPAIR COMPLETED	2011-12-15 15:38:28	UNKNOWN	OTHER	CATEGORY 1
3	1328	2006-09-11 00:00:00	SERVICE	REPAIR COMPLETED	2011-12-15 15:38:28	UNKNOWN	OTHER	CATEGORY 1
4	1308	2000-01-27 00:00:00	SERVICE	REPAIR COMPLETED	2011-12-15 15:38:28	UNKNOWN	OTHER	CATEGORY 1

5 rows × 21 columns

In [22]:

```
breaks_with_soil = gpd.sjoin(
    breaks_points,
    soil_small,
    how="left",
    predicate="within"
)

print(breaks_with_soil.shape)
breaks_with_soil.head()
```

```
(2994, 34)
```

Out[22]:

	Wat Break Incident ID	Incident date	Type of Asset Broken	Current status of the break	Status last updated date	Nature of Break	Apparent cause of break	Categorization of the Break
0	2252	2017-12-01 15:15:00	MAIN	REPAIR COMPLETED	2017-12-02 00:00:00	CORROSION AND FITTING/JOINT	AGE	CATEGORY 2
1	1311	2001-03-26 00:00:00	SERVICE	REPAIR COMPLETED	2011-12-15 15:38:28	UNKNOWN	OTHER	CATEGORY 1
2	1325	2006-09-06 00:00:00	SERVICE	REPAIR COMPLETED	2011-12-15 15:38:28	UNKNOWN	OTHER	CATEGORY 1
3	1328	2006-09-11 00:00:00	SERVICE	REPAIR COMPLETED	2011-12-15 15:38:28	UNKNOWN	OTHER	CATEGORY 1
4	1308	2000-01-27 00:00:00	SERVICE	REPAIR COMPLETED	2011-12-15 15:38:28	UNKNOWN	OTHER	CATEGORY 1

5 rows × 34 columns

In [23]:

```
soil_match_count = breaks_with_soil["SOIL_NAME1"].notna().sum()
soil_total = len(breaks_with_soil)
soil_match_pct = round(soil_match_count / soil_total * 100, 2)

print("Break rows with soil match:", soil_match_count)
print("Total break rows:", soil_total)
print("Soil match %:", soil_match_pct)
```

```
Break rows with soil match: 2994
Total break rows: 2994
Soil match %: 100.0
```

In [24]:

```
#Inspect exact raw values with repr(), so spaces/newlines become visible
sample_vals = breaks_with_soil["ATEXTURE1"].dropna().astype(str).unique()[:50]

for v in sample_vals:
    print(repr(v))
```

```
'NA'
'FSL'
'GL'
'L'
'SIL'
'SL'
```

In [25]:

```
#Clean ATEXTURE1
breaks_with_soil["ATEXTURE1_CLEAN"] = (
    breaks_with_soil["ATEXTURE1"]
    .astype(str)
    .str.strip()
    .str.upper()
)

#Turn obvious fake values back to missing
breaks_with_soil["ATEXTURE1_CLEAN"] = breaks_with_soil["ATEXTURE1_CLEAN"].replace({
    "NAN": pd.NA,
    "": pd.NA
})
```

```
print(breaks_with_soil["ATEXTURE1_CLEAN"].value_counts(dropna=False).head(30))
```

```
ATEXTURE1_CLEAN
NA     2950
FSL      23
SL       11
GL        7
L         2
SIL       1
Name: count, dtype: int64
```

In [26]:

```
for col in ["SOIL_NAME1", "DRAINAGE1", "HYDRO1", "PARNT_MAT1", "ATEXTURE1_CLEAN"]:
    print(f"\n=== {col} ===")
    print(breaks_with_soil[col].isna().mean() * 100)
    print(breaks_with_soil[col].value_counts(dropna=False).head(20))
```

```
=== SOIL_NAME1 ===
0.0
SOIL_NAME1
URBAN                        2943
WATERLOO FINE SANDY LOAM       23
BURFORD GRAVELLY LOAM           7
LISBON SANDY LOAM               7
WATER                           4
BUILT UP AREA                   3
ELMIRA LOAM                     2
FOX SANDY LOAM                  2
ST CLEMENTS SILT LOAM           1
CALEDON SANDY LOAM              1
BOOKTON SANDY LOAM              1
Name: count, dtype: int64

=== DRAINAGE1 ===
0.0
DRAINAGE1
NA    2946
W       42
WA       4
P        2
Name: count, dtype: int64

=== HYDRO1 ===
0.0
HYDRO1
N    2950
A      40
C       3
B       1
Name: count, dtype: int64

=== PARNT_MAT1 ===
100.0
PARNT_MAT1
None    2994
Name: count, dtype: int64

=== ATEXTURE1_CLEAN ===
0.0
ATEXTURE1_CLEAN
NA     2950
FSL      23
SL       11
GL        7
L         2
SIL       1
Name: count, dtype: int64
```

In [27]:

```python
#Main soil texture distribution from break-level enriched dataset
texture_counts = breaks_with_soil["ATEXTURE1"].value_counts(dropna=False)

print("Top soil texture categories in break records:")
display(texture_counts.head(20))

#Percentage version
texture_pct = (
    breaks_with_soil["ATEXTURE1"]
    .value_counts(normalize=True, dropna=False)
    .mul(100)
    .round(2)
)
```

```
print("Top soil texture categories (%) in break records:")
display(texture_pct.head(20))
```

```
Top soil texture categories in break records:
ATEXTURE1
NA     2950
FSL      23
SL       11
GL        7
L         2
SIL       1
Name: count, dtype: int64
Top soil texture categories (%) in break records:
ATEXTURE1
NA     98.53
FSL     0.77
SL      0.37
GL      0.23
L       0.07
SIL     0.03
Name: proportion, dtype: float64
```

In [28]:

```
import numpy as np

def simplify_texture(val):
    if pd.isna(val):
        return "Unknown"

    v = str(val).strip().upper()

    # Organic
    if v in ["ORG", "PEAT", "PT", "MUCK", "M", "O"]:
        return "Organic / Peat"

    # Sand-dominant textures
    elif v in ["S", "FS", "LS", "SL", "CSL", "FSL", "LFS", "LVFS", "VFSL", "GS", "GSL
        return "Sand-heavy"

    # Clay-dominant textures
    elif v in ["C", "CL", "SIC", "SICL"]:
        return "Clay-heavy"

    # Loam textures
    elif v in ["L", "GL"]:
        return "Loam / mixed"

    # Silt-dominant textures
    elif v in ["SIL"]:
        return "Silt-heavy"

    # Gravel / variable / other special
    elif v in ["GRAV", "VAR", "R"]:
        return "Other / Mixed"

    else:
        return "Other / Mixed"

breaks_with_soil["ATEXTURE1_CLEAN"] = (
    breaks_with_soil["ATEXTURE1"]
    .astype(str)
    .str.strip()
    .str.upper()
    .replace({"NAN": pd.NA, "": pd.NA})
)

breaks_with_soil["SOIL_TEXTURE_GROUP"] = breaks_with_soil["ATEXTURE1_CLEAN"].apply(si
```

```
group_counts = breaks_with_soil["SOIL_TEXTURE_GROUP"].value_counts(dropna=False)
group_pct = breaks_with_soil["SOIL_TEXTURE_GROUP"].value_counts(normalize=True, dropn

print("Broad soil groups in break records:")
display(group_counts)

print("Broad soil groups in break records (%):")
display(group_pct)
```

```
Broad soil groups in break records:
SOIL_TEXTURE_GROUP
Other / Mixed    2950
Sand-heavy         34
Loam / mixed        9
Silt-heavy          1
Name: count, dtype: int64
Broad soil groups in break records (%):
SOIL_TEXTURE_GROUP
Other / Mixed    98.53
Sand-heavy        1.14
Loam / mixed      0.30
Silt-heavy        0.03
Name: proportion, dtype: float64
```

In [29]:

```
#CANSIS dataset
#File paths
slc_shp_path = "../data/ca_all_slc_v3r2.shp"
cmp_path = "../data/ca_all_slc_v3r2_cmp.dbf"
snt_path = "../data/soil_name_canada_v2r20231107.dbf"

#Load SLC polygon layer
slc_gdf = gpd.read_file(slc_shp_path)

print("SLC polygons shape:", slc_gdf.shape)
print("SLC CRS:", slc_gdf.crs)
print("SLC columns:")
print(slc_gdf.columns.tolist())

#Load CMP table
cmp_df = pd.read_csv(cmp_path) if cmp_path.endswith(".csv") else gpd.read_file(cmp_pa

#If gpd.read_file on DBF returns a GeoDataFrame-like table, convert to plain DataFram
cmp_df = pd.DataFrame(cmp_df)

print("\nCMP shape:", cmp_df.shape)
print("CMP columns:")
print(cmp_df.columns.tolist())

#Load Soil Name Table (optional)
try:
    snt_df = gpd.read_file(snt_path)
    snt_df = pd.DataFrame(snt_df)
    print("\nSNT shape:", snt_df.shape)
    print("SNT columns:")
    print(snt_df.columns.tolist())
except Exception as e:
    snt_df = None
    print("\nCould not load Soil Name Table yet:", e)
```

```
SLC polygons shape: (12353, 5)
SLC CRS: EPSG:4269
SLC columns:
['AREA', 'PERIMETER', 'POLY_ID', 'ECO_ID', 'geometry']

CMP shape: (20425, 12)
CMP columns:
['POLY_ID', 'CMP', 'PERCENT', 'SLOPE', 'STONE', 'LOCSF', 'PROVINCE', 'SOIL_CODE', 'MOD
IFIER', 'PROFILE', 'SOIL_ID', 'CMP_ID']

SNT shape: (14581, 26)
SNT columns:
['SOIL_ID', 'PROVINCE', 'SOIL_CODE', 'MODIFIER', 'PROFILE', 'SOILNAME', 'KIND', 'WATER
TBL', 'ROOTRESTRI', 'RESTR_TYPE', 'DRAINAGE', 'PMTEX1', 'PMTEX2', 'PMTEX3', 'PMCHEM1',
'PMCHEM2', 'PMCHEM3', 'MDEP1', 'MDEP2', 'MDEP3', 'ORDER2', 'G_GROUP2', 'S_GROUP2', 'OR
DER3', 'G_GROUP3', 'S_GROUP3']
```

In [30]:

```python
#Approximate Kitchener-Waterloo bounding box in EPSG:4326
min_lon, max_lon = -80.65, -80.30
min_lat, max_lat = 43.35, 43.55

#Clip SLC polygons to Kitchener-Waterloo area
slc_kw = slc_gdf.cx[min_lon:max_lon, min_lat:max_lat].copy()

print("Kitchener-Waterloo SLC polygons:", slc_kw.shape)
display(slc_kw.head())
```

```
Kitchener-Waterloo SLC polygons: (7, 5)
```

	AREA	PERIMETER	POLY_ID	ECO_ID	geometry
12242	0.095838	2.794653	560003	560	POLYGON ((-80.05926 43.76209, -80.0595 43.7609...
12250	0.219033	4.454710	557005	557	POLYGON ((-80.65324 43.607, -80.65165 43.60635...
12251	0.062583	2.943434	560004	560	POLYGON ((-80.14708 43.57857, -80.14679 43.577...
12255	0.064518	1.731529	564005	564	POLYGON ((-80.13199 43.56943, -80.12815 43.569...
12258	0.015487	0.760011	560005	560	POLYGON ((-80.6308 43.57209, -80.63043 43.5720...

In [31]:

```python
#Keep useful CMP fields only
cmp_keep_cols = [
    "POLY_ID",
    "CMP",
    "PERCENT",
    "SLOPE",
    "STONE",
    "LOCSF",
    "PROVINCE",
    "SOIL_CODE",
    "MODIFIER",
    "PROFILE",
    "SOIL_ID",
    "CMP_ID"
]

cmp_small = cmp_df[[c for c in cmp_keep_cols if c in cmp_df.columns]].copy()

print("CMP small shape:", cmp_small.shape)
display(cmp_small.head())
```

```
#Ensure PERCENT is numeric
cmp_small["PERCENT"] = pd.to_numeric(cmp_small["PERCENT"], errors="coerce")

#Keep dominant component per POLY_ID
cmp_dominant = (
    cmp_small
    .sort_values(["POLY_ID", "PERCENT"], ascending=[True, False])
    .drop_duplicates(subset=["POLY_ID"], keep="first")
    .copy()
)

print("Dominant CMP shape:", cmp_dominant.shape)
display(cmp_dominant.head())
```

```
CMP small shape: (20425, 12)
```

	POLY_ID	CMP	PERCENT	SLOPE	STONE	LOCSF	PROVINCE	SOIL_CODE	MODIFIER	PROFILE
0	242021	1	40	A	U	D	AB	BUF	gl~~~	N
1	242021	2	30	A	N	U	AB	KEG	~~~~~	N
2	242021	3	30	A	N	F13	AB	MLD	~~~~~	N
3	242022	1	40	A	N	B07	AB	MKW	~~~~~	N
4	242022	2	30	A	N	F13	AB	MLD	~~~~~	N

```
Dominant CMP shape: (4579, 12)
```

	POLY_ID	CMP	PERCENT	SLOPE	STONE	LOCSF	PROVINCE	SOIL_CODE	MODIFIER	PROFIL
0	242021	1	40	A	U	D	AB	BUF	gl~~~	
3	242022	1	40	A	N	B07	AB	MKW	~~~~~	
6	242023	1	40	A	N	B07	AB	MKW	~~~~~	
9	242024	1	60	A	N	B07	AB	MKW	~~~~~	
12	242025	1	30	A	S	U	AB	COS	~~~~~	

In [32]:

```
#breaks_test should already contain longitude / latitude from earlier conversion
breaks_points = gpd.GeoDataFrame(
    breaks_test.copy(),
    geometry=gpd.points_from_xy(breaks_test["longitude"], breaks_test["latitude"]),
    crs="EPSG:4326"
)

print("Break points shape:", breaks_points.shape)
display(breaks_points[["Wat Break Incident ID", "Street", "longitude", "latitude"]].h
```

```
Break points shape: (2994, 21)
```

	Wat Break Incident ID	Street	longitude	latitude
0	2252	LANCASTER ST W	-80.484001	43.462931
1	1311	CLOVERDALE CRES	-80.515071	43.422734
2	1325	WREN CRES	-80.439808	43.445058
3	1328	GREENBROOK DR	-80.510856	43.426469
4	1308	MONTGOMERY RD	-80.457516	43.443192

In [33]:
```
#Keep only minimal polygon fields first
slc_poly_keep = ["POLY_ID", "ECO_ID", "geometry"]
slc_kw_small = slc_kw[[c for c in slc_poly_keep if c in slc_kw.columns]].copy()

#Spatial join
breaks_slc = gpd.sjoin(
    breaks_points,
    slc_kw_small,
    how="left",
    predicate="within"
)

print("Breaks joined to SLC polygons:", breaks_slc.shape)
print("Break rows with POLY_ID:", breaks_slc["POLY_ID"].notna().sum())
print("Break rows without POLY_ID:", breaks_slc["POLY_ID"].isna().sum())

display(breaks_slc[["Wat Break Incident ID", "Street", "POLY_ID", "ECO_ID"]].head())
```

```
Breaks joined to SLC polygons: (2994, 24)
Break rows with POLY_ID: 2994
Break rows without POLY_ID: 0
```

```
/var/folders/bk/sln2b49n47g24cwmlq18xqhc0000gn/T/ipykernel_7879/3793365015.py:6: UserW
arning: CRS mismatch between the CRS of left geometries and the CRS of right geometrie
s.
Use `to_crs()` to reproject one of the input geometries to match the CRS of the other.

Left CRS: EPSG:4326
Right CRS: EPSG:4269

  breaks_slc = gpd.sjoin(
```

	Wat Break Incident ID	Street	POLY_ID	ECO_ID
0	2252	LANCASTER ST W	560004	560
1	1311	CLOVERDALE CRES	560005	560
2	1325	WREN CRES	560006	560
3	1328	GREENBROOK DR	560005	560
4	1308	MONTGOMERY RD	560006	560

In [34]:
```
#Fix CRS mismatch before spatial join
#breaks_points = EPSG:4326
#slc_kw_small = EPSG:4269
#Reproject SLC polygons to match breaks
slc_kw_small = slc_kw_small.to_crs(breaks_points.crs)

print("Break CRS:", breaks_points.crs)
print("SLC CRS after reprojection:", slc_kw_small.crs)

#Spatial join again after CRS fix
breaks_slc = gpd.sjoin(
    breaks_points,
    slc_kw_small,
    how="left",
    predicate="within"
)

print("Breaks joined to SLC polygons:", breaks_slc.shape)
print("Break rows with POLY_ID:", breaks_slc["POLY_ID"].notna().sum())
print("Break rows without POLY_ID:", breaks_slc["POLY_ID"].isna().sum())

display(breaks_slc[["Wat Break Incident ID", "Street", "POLY_ID", "ECO_ID"]].head())
```

```
Break CRS: EPSG:4326
SLC CRS after reprojection: EPSG:4326
Breaks joined to SLC polygons: (2994, 24)
Break rows with POLY_ID: 2994
Break rows without POLY_ID: 0
```

	Wat Break Incident ID	Street	POLY_ID	ECO_ID
0	2252	LANCASTER ST W	560004	560
1	1311	CLOVERDALE CRES	560005	560
2	1325	WREN CRES	560006	560
3	1328	GREENBROOK DR	560005	560
4	1308	MONTGOMERY RD	560006	560

In [35]:

```python
#Keep only useful CMP columns
cmp_keep_cols = [
    "POLY_ID",
    "CMP",
    "PERCENT",
    "SLOPE",
    "STONE",
    "LOCSF",
    "PROVINCE",
    "SOIL_CODE",
    "MODIFIER",
    "PROFILE",
    "SOIL_ID",
    "CMP_ID"
]

cmp_small = cmp_df[[c for c in cmp_keep_cols if c in cmp_df.columns]].copy()

#Make sure PERCENT is numeric
cmp_small["PERCENT"] = pd.to_numeric(cmp_small["PERCENT"], errors="coerce")

# Keep dominant component only
# For each POLY_ID, keep the row with highest PERCENT
cmp_dominant = (
    cmp_small
    .sort_values(["POLY_ID", "PERCENT"], ascending=[True, False])
    .drop_duplicates(subset=["POLY_ID"], keep="first")
    .copy()
)

print("Dominant CMP shape:", cmp_dominant.shape)
display(cmp_dominant.head())
```

```
Dominant CMP shape: (4579, 12)
```

	POLY_ID	CMP	PERCENT	SLOPE	STONE	LOCSF	PROVINCE	SOIL_CODE	MODIFIER	PROFIL
0	242021	1	40	A	U	D	AB	BUF	gl~~~	
3	242022	1	40	A	N	B07	AB	MKW	~~~~~	
6	242023	1	40	A	N	B07	AB	MKW	~~~~~	
9	242024	1	60	A	N	B07	AB	MKW	~~~~~	
12	242025	1	30	A	S	U	AB	COS	~~~~~	

In [36]:
```
#Attach dominant CMP info using POLY_ID
breaks_slc_cmp = breaks_slc.merge(
    cmp_dominant,
    on="POLY_ID",
    how="left"
)

print("Breaks + dominant CMP shape:", breaks_slc_cmp.shape)

display(
    breaks_slc_cmp[
        ["Wat Break Incident ID", "Street", "POLY_ID", "SOIL_ID", "SOIL_CODE", "PERCE
    ].head()
)
```

```
Breaks + dominant CMP shape: (2994, 35)
```

	Wat Break Incident ID	Street	POLY_ID	SOIL_ID	SOIL_CODE	PERCENT	SLOPE	STONE
0	2252	LANCASTER ST W	560004	ONBUF~~~~~A	BUF	37	B	S
1	1311	CLOVERDALE CRES	560005	ONWTO~~~~~A	WTO	29	B	N
2	1325	WREN CRES	560006	ONFOX~~~~~A	FOX	20	B	N
3	1328	GREENBROOK DR	560005	ONWTO~~~~~A	WTO	29	B	N
4	1308	MONTGOMERY RD	560006	ONFOX~~~~~A	FOX	20	B	N

In [37]:
```
#Keep only useful columns from soil name table
snt_keep_cols = [
    "SOIL_ID",
    "SOILNAME",
    "DRAINAGE",
    "PMTEX1",
    "PMTEX2",
    "PMTEX3",
    "ORDER3",
    "G_GROUP3",
    "S_GROUP3"
]

snt_small = snt_df[[c for c in snt_keep_cols if c in snt_df.columns]].copy()

#Merge SNT onto break-level data using SOIL_ID
breaks_slc_cmp_named = breaks_slc_cmp.merge(
    snt_small,
    on="SOIL_ID",
    how="left"
)

print("Breaks + CMP + SNT shape:", breaks_slc_cmp_named.shape)

display(
    breaks_slc_cmp_named[
        ["SOIL_ID", "SOIL_CODE", "SOILNAME", "DRAINAGE", "PMTEX1", "ORDER3", "G_GROUP
    ].drop_duplicates().head(20)
)
```

```
Breaks + CMP + SNT shape: (2994, 43)
```

	SOIL_ID	SOIL_CODE	SOILNAME	DRAINAGE	PMTEX1	ORDER3	G_GROUP3	S_GRO
0	ONBUF~~~~~A	BUF	BURFORD	W	VC	LU	GBL	
1	ONWTO~~~~~A	WTO	WATERLOO	W	VC	LU	GBL	
2	ONFOX~~~~~A	FOX	FOX	W	VC	LU	GBL	
2558	ONGUP~~~~~A	GUP	GUELPH	W	M	LU	GBL	

In [38]:
```
#Check missingness of key soil fields
soil_check_cols = [
    "POLY_ID",
    "SOIL_ID",
    "SOIL_CODE",
    "SOILNAME",
    "DRAINAGE",
    "PMTEX1",
    "SLOPE",
    "STONE"
]

soil_missing_summary = pd.DataFrame({
    "missing_count": breaks_slc_cmp_named[soil_check_cols].isnull().sum(),
    "missing_ratio_pct": (breaks_slc_cmp_named[soil_check_cols].isnull().mean() * 100
}).sort_values("missing_ratio_pct", ascending=False)

display(soil_missing_summary)
```

	missing_count	missing_ratio_pct
POLY_ID	0	0.0
SOIL_ID	0	0.0
SOIL_CODE	0	0.0
SOILNAME	0	0.0
DRAINAGE	0	0.0
PMTEX1	0	0.0
SLOPE	0	0.0
STONE	0	0.0

In [39]:
```
for col in ["SOIL_CODE", "SOILNAME", "DRAINAGE", "PMTEX1", "SLOPE", "STONE"]:
    print(f"\n=== {col} ===")
    print(breaks_slc_cmp_named[col].value_counts(dropna=False).head(20))
```

```
=== SOIL_CODE ===
SOIL_CODE
FOX    1718
BUF     701
WTO     572
GUP       3
Name: count, dtype: int64

=== SOILNAME ===
SOILNAME
FOX         1718
BURFORD      701
WATERLOO     572
GUELPH         3
Name: count, dtype: int64

=== DRAINAGE ===
DRAINAGE
W    2994
Name: count, dtype: int64

=== PMTEX1 ===
PMTEX1
VC    2991
M        3
Name: count, dtype: int64

=== SLOPE ===
SLOPE
B    2994
Name: count, dtype: int64

=== STONE ===
STONE
N    2290
S     704
Name: count, dtype: int64
```

In [40]:

```python
def texture_group(val):
    if pd.isna(val):
        return "Unknown"

    v = str(val).strip().upper()

    # Coarse / sandy / variable coarse
    if v in ["S", "FS", "LS", "LFS", "LVFS", "GS", "VC"]:
        return "Sand-dominant"

    # Loam / sandy loam / mixed loam
    elif v in ["L", "SL", "FSL", "VFSL", "GL", "GSL"]:
        return "Loam / sandy-loam"

    # Fine / clay / silt / clay-loam style
    elif v in ["CL", "C", "SIL", "SICL", "SIC", "CSL", "M"]:
        return "Silt / clay-dominant"

    else:
        return "Other / Mixed"

breaks_slc_cmp_named["TEXTURE_GROUP"] = breaks_slc_cmp_named["PMTEX1"].apply(texture_

print("\n=== TEXTURE_GROUP ===")
print(breaks_slc_cmp_named["TEXTURE_GROUP"].value_counts(dropna=False))
print(
    breaks_slc_cmp_named["TEXTURE_GROUP"]
    .value_counts(normalize=True, dropna=False)
```

```
    .mul(100)
    .round(2)
)
```

```
=== TEXTURE_GROUP ===
TEXTURE_GROUP
Sand-dominant           2991
Silt / clay-dominant       3
Name: count, dtype: int64
TEXTURE_GROUP
Sand-dominant           99.9
Silt / clay-dominant     0.1
Name: proportion, dtype: float64
```